

1 Task 3 — Graph Embedding over Homogeneous Structures

1.1 Approach

All four notebook TODOs are implemented; the provided scaffolding (`load_data`, `adjacency_lookups`, `evaluate_ranking`, `breakdown_by_source_degree`, `run_experiment`, `comparison_table`, ...) is used verbatim and `RANDOM_SEED = 0`. Loading reproduces the published statistics exactly: 7,624 users, 19,464 observed edges, 666 isolated users, mean degree 5.106, max degree 153, 734 components, and $4,171 \times 21$ validation/test queries.

sample_walks. A strategy is nothing but a transition rule; the *walk budget is identical for every strategy* (10 walks of length 40 from each of the 6,958 non-isolated users, giving 69,580 walks in every run). The 666 isolated users return [], as the docstring permits, and therefore keep a zero embedding row. Seven rules are compared:

strategy	transition rule $P(x \mid \cdot)$	hypothesis being tested
DeepWalk	uniform over $N(v)$	baseline: unbiased 1st-order walk
node2vec	$\propto 1/p, 1, 1/q$ by distance to t	$q < 1$ outward (homophily), $q > 1$ inward (structural equivalence)
RWR	uniform, teleport to start w.p. r	a strongly <i>personalised</i> context should sharpen pairwise proximity, at the cost of global community signal
deg-corrected	$\propto \deg(x)^{-\alpha}$	uniform walks over-visit hubs; equalising visit counts should lift the low-degree source buckets
triadic	$\propto 1 + N(v) \cap N(x) $	preferring triangle-closing steps keeps the walk inside a community, i.e. maximal homophily

RWR, deg-corrected and triadic are my own additions. node2vec uses the standard $O(1)$ rejection sampler rather than materialising alias tables, which keeps a second-order walk at 1.3s for the whole corpus.

train_embedding. Skip-Gram with negative sampling (`gensim.Word2Vec`, `sg=1`) over the walk corpus. I set `sample=0`, i.e. *no* frequent-token subsampling: subsampling would itself down-weight hubs and so half-implement the deg-corrected strategy, confounding that comparison. `workers=1` makes every run bit-exactly reproducible (gensim’s multi-threaded training is not deterministic).

Downstream models — held fixed across all strategies. As the notebook demands, `LinkPredictor` and `CountryClassifier` are the same code for every row of the comparison table; the embedding is the only thing that changes. Both ℓ_2 -normalise Z row-wise (zero rows stay zero), standardise, and fit a logistic regression. A pair (u, v) becomes

$$\phi(u, v) = \left[\underbrace{z_u \odot z_v}_{\text{Hadamard}}, \underbrace{\|z_u - z_v\|_2}_{\text{abs. diff.}}, \cos(z_u, z_v), z_u^\top z_v, \|z_u - z_v\|_2 \right] \in \mathbb{R}^{2d+3},$$

the node2vec paper’s two best binary operators concatenated with three scalar similarities. `score` returns the continuous `decision_function`, so higher always means ”more likely an edge”.

Table 1: *
Fixed hyper-parameters (identical in every run).

walks per node	10	embedding dim. d	128
walk length	40	Skip-Gram window	5
corpus size	69,580 walks	negative samples	5
seed	0 (also 1, 2)	epochs	5
LP regulariser	$C \in \{0.1, 1, 10\}$, chosen by val. MRR	subsampling	off (sample=0)
CC regulariser	$C \in \{0.1, 1, 10, 100\}$, chosen by val. micro-F1	solver	lbfgs

1.2 Implementation

Only the code I wrote is listed; the provided scaffolding is omitted.

```
def sample_walks(self, G, start):
    """'num_walks' walks of length 'walk_length', all beginning at 'start'."""
    adj, deg = adjacency_lookups(G)
    if deg[start] == 0: # isolated node -> no walks (allowed)
        return []
    rng, L, R, strat = self._rng, self.walk_length, self.num_walks, self.strategy

    if strat == "node2vec": # 2nd-order, O(1) rejection sampling
        NBR = self._nbr_lists(G) # cached sorted neighbour lists
        p, q = self.params["p"], self.params["q"]
        inv_p, inv_q, wmax = 1/p, 1/q, max(1/p, 1.0, 1/q)
        walks = []
        for _ in range(R):
            walk = [start, rng.choice(NBR[start])] # 1st step unbiased
            while len(walk) < L:
                cur, prev = walk[-1], walk[-2]
                cn, prev_nbrs = NBR[cur], adj[prev]
                while True:
                    x = rng.choice(cn)
                    w = inv_p if x == prev else (1.0 if x in prev_nbrs else inv_q)
                    if rng.random() * wmax < w:
                        break
                walk.append(x)
            walks.append(walk)
        return walks

    if strat in ("degcorr", "triadic"): # 1st-order, edge-weighted
        tables = self._weighted_tables(G) # per node: (nbrs, cumulative weights)
        walks = [] # degcorr: w(x) = deg(x)**-alpha
        for _ in range(R): # triadic: w(x) = 1 + |N(cur) & N(x)|
            walk = [start]
            while len(walk) < L:
                nbrs, cum = tables[walk[-1]]
                walk.append(rng.choices(nbrs, cum_weights=cum, k=1)[0])
            walks.append(walk)
        return walks

    NBR = self._nbr_lists(G) # "uniform" (DeepWalk) and "rwr"
    restart = self.params.get("restart", 0.0) if strat == "rwr" else 0.0
    walks = []
    for _ in range(R):
        walk = [start]
        while len(walk) < L:
            if restart and rng.random() < restart:
                walk.append(start) # teleport home, then carry on
            else:
                walk.append(rng.choice(NBR[walk[-1]]))
        walks.append(walk)
```

```
return walks
```

Listing 1: TODO 1 — sample_walks

```
def train_embedding(self, walks, n_nodes):
    d = self.params.get("dim", 128)
    model = Word2Vec(sentences=[[str(v) for v in w] for w in walks],
                     vector_size=d, window=self.params.get("window", 5),
                     sg=1, hs=0, negative=self.params.get("negative", 5),
                     epochs=self.params.get("epochs", 5),
                     min_count=0, sample=0, seed=self.seed, workers=1)
    Z = np.zeros((n_nodes, d), dtype=float) # nodes never walked keep a zero row
    Z[[int(k) for k in model.wv.index_to_key]] = model.wv.vectors
    return Z
```

Listing 2: TODO 2 — train_embedding

```
def _unit(Z): # row-wise L2 norm; all-zero rows stay zero
    n = np.linalg.norm(Z, axis=1, keepdims=True)
    return Z / np.where(n == 0.0, 1.0, n)

def _pair_features(Z, pairs): # (n_q, n_cand, 2) -> (n_q*n_cand, 2d+3)
    u, v = Z[pairs[:, 0]].reshape(-1, Z.shape[1]), Z[pairs[:, 1]].reshape(-1, Z.shape[1])
    had, dif = u * v, np.abs(u - v)
    dot = had.sum(1, keepdims=True)
    l2 = np.linalg.norm(u - v, axis=1, keepdims=True)
    cos = dot / np.maximum(np.linalg.norm(u, axis=1, keepdims=True)
                          * np.linalg.norm(v, axis=1, keepdims=True), 1e-12)
    return np.hstack([had, dif, cos, dot, l2])

class LinkPredictor:
    C_GRID = (0.1, 1.0, 10.0)

    def fit(self, Z, pairs_train, y_train, pairs_val, y_val):
        Zn = _unit(np.asarray(Z, dtype=float))
        Xtr = _pair_features(Zn, pairs_train)
        self.scaler = StandardScaler().fit(Xtr)
        Xtr, ytr = self.scaler.transform(Xtr), np.asarray(y_train).ravel()
        Xva = self.scaler.transform(_pair_features(Zn, pairs_val))
        nq, nc = pairs_val.shape[:2]
        pos_val = np.argmax(np.asarray(y_val), axis=1)

        best = (-1.0, None, None)
        for C in self.C_GRID: # model selection on validation MRR
            clf = LogisticRegression(C=C, max_iter=2000, random_state=RANDOM_SEED)
            clf.fit(Xtr, ytr)
            s = clf.decision_function(Xva).reshape(nq, nc)
            rank = 1 + (s > s[np.arange(nq), pos_val][:, None]).sum(1)
            mrr = float(np.mean(1.0 / rank))
            if mrr > best[0]:
                best = (mrr, clf, C)
        self.val_mrr_, self.clf, self.C_ = best
        return self

    def score(self, Z, pairs): # continuous, higher = more likely
        X = self.scaler.transform(_pair_features(_unit(np.asarray(Z, float)), pairs))
        return self.clf.decision_function(X).reshape(pairs.shape[:2])
```

Listing 3: TODO 3a — LinkPredictor (and the pair feature map)

```
class CountryClassifier:
    C_GRID = (0.1, 1.0, 10.0, 100.0)

    def fit(self, Z_train, y_train, Z_val, y_val):
        Xtr = _unit(np.asarray(Z_train, dtype=float))
        self.scaler = StandardScaler().fit(Xtr)
        Xtr = self.scaler.transform(Xtr)
```

```

Xva = self.scaler.transform(_unit(np.asarray(Z_val, dtype=float)))
best = (-1.0, None, None)
for C in self.C_GRID: # selection on validation micro-F1
    clf = LogisticRegression(C=C, max_iter=5000, random_state=RANDOM_SEED)
    clf.fit(Xtr, y_train)
    acc = float((clf.predict(Xva) == np.asarray(y_val)).mean())
    if acc > best[0]:
        best = (acc, clf, C)
self.val_micro_f1_, self.clf, self.C_ = best
return self

def predict(self, Z):
    return self.clf.predict(self.scaler.transform(_unit(np.asarray(Z, float))))

```

Listing 4: TODO 3b — CountryClassifier